



INDIA.RUNS

Build what next India runs on

Team Name : Reflex

Team Leader Name : Gnana Venkata Reddy Datla

Problem Statement :Recruiters manually sift through hundreds of profiles for every role, relying on keyword filters that miss qualified candidates and surface unqualified ones. There is no automated, context-aware system that understands a job description semantically and ranks candidates the way an expert recruiter would – by evaluating career trajectory, skill depth, behavioral signals, and role fit holistically.

Solution Overview

- An offline, CPU-only AI ranking engine that reads a Job Description, extracts semantic requirements, and scores 100,000 candidates in under 40 seconds – producing a trusted shortlist of the top 100 candidates with human-readable explanations for every rank.
- Traditional systems match keywords. Our system understands context – it uses sentence-transformer embeddings for semantic similarity, a multi-signal weighted scoring model (skills, seniority, trajectory, behavioral signals), a honeypot detector to exclude fake profiles, and a dynamic weight normalizer that fairly handles incomplete profiles.

JD Understanding & Candidate Evaluation

- Role: Senior AI Engineer (Founding Team). Key requirements: 5+ years ML/AI experience, hands-on with LLMs, RAG, vector search, Python, and cloud deployment. Leadership potential and startup mindset required.
- We evaluate 7 signals:
 - (1) Semantic skill similarity via embeddings
 - (2) Seniority classification from title + years
 - (3) Career trajectory / growth over time
 - (4) Platform engagement – posts, endorsements
 - (5) Recency of last role
 - (6) Certification relevance
 - (7) Behavioral red-flag / honeypot detection. No keyword matching – every signal is scored numerically and combined into a weighted composite score.

Ranking Methodology

- Step 1 → Load all 100,000 candidates from JSONL.
Step 2 → Extract features per candidate (skills, title, years, activity).
Step 3 → Score each candidate on 7 weighted dimensions.
Step 4 → Sort by composite score descending with candidate_id tie-breaking.
Step 5 → Output top 100 to CSV.
- Sentence-Transformers (all-MiniLM-L6-v2) for embedding-based skill similarity. Rule-based title classifier for seniority gating. Weighted linear combination for final composite score.
Deterministic honeypot filters for impossible tenure and duplicate signals.
- Final Score = $(0.35 \times \text{Skill Similarity}) + (0.25 \times \text{Seniority}) + (0.15 \times \text{Trajectory}) + (0.10 \times \text{Behavioral}) + (0.10 \times \text{Recency}) + (0.05 \times \text{Certifications})$. Weights are normalized dynamically when a signal is missing so incomplete profiles are not unfairly penalized.

Explainability & Data Validation

- Every candidate in the output CSV includes a plain-English reasoning column explaining why they ranked where they did – e.g., "Strong LLM + RAG skill match, 7 years senior trajectory, high platform engagement." No black-box outputs.
- All scores are derived purely from structured candidate data – no LLM generation is used for scoring decisions. Reasoning strings are templated directly from actual numeric scores, so every justification is fully traceable to measurable signals.
- A dedicated honeypot detector flags and excludes profiles with impossible employment timelines, suspiciously uniform skill lists, and missing critical fields – all before scoring begins, keeping the shortlist clean and trustworthy.

End-to-End Workflow

- Step 1 → JD is parsed and key role requirements are extracted (skills, seniority, domain).
- Step 2 → All 100,000 candidate profiles are loaded from candidates.jsonl.
- Step 3 → Honeypot detector filters out fake/suspicious profiles before scoring.
- Step 4 → Each candidate is scored across 7 signals: skill similarity, seniority, trajectory, behavioral engagement, recency, certifications, and red-flag detection.
- Step 5 → Dynamic weight normalization adjusts scores for candidates with missing fields.
- Step 6 → All candidates are ranked by composite score. Top 100 are selected.
- Step 7 → Output is written to submission.csv with rank, score, and plain-English reasoning per candidate.
- Step 8 → Results are viewable via the live Streamlit sandbox for recruiter review.

System Architecture

Input Layer:

- candidates.jsonl (100,000 profiles) + job_description (hardcoded in jd_config.py)

Feature Extraction (features.py):

- Skill parser, title classifier, tenure calculator, activity scorer

Scoring Engine (scoring.py):

- 7-signal weighted composite score
- Dynamic normalization for incomplete profiles

Honeypot Filter (rank.py):

- Removes impossible timelines, duplicate signals, missing critical fields

Ranking & Output (rank.py):

- Top-100 sorted by score → submission.csv with reasoning column

Deployment:

- Streamlit sandbox (sandbox/app.py) hosted on Streamlit Cloud
- GitHub: https://github.com/datlavenkatareddy/REDROB_RANKER

Results & Performance

- Top-ranked candidates are genuine Senior AI/ML Engineers with 5–10 years experience, strong LLM + RAG + vector search skills, and high platform engagement. Honeypot profiles (fake, padded, or impossible timelines) are correctly excluded before scoring. The output CSV passes the official hackathon validator with exactly 100 ranked entries, sorted correctly, with no formatting errors.
- Full pipeline (100,000 candidates) completes in under 40 seconds on a single CPU core with no GPU required. Memory usage stays under 2 GB. The system runs fully offline – no API calls, no internet dependency. Optionally, a precomputed embedding cache (embed_sims.pkl) can be loaded to further accelerate re-runs.

Technologies Used

Python 3.10 – Core language for the entire pipeline.

Sentence-Transformers (all-MiniLM-L6-v2) – Semantic skill similarity via cosine embeddings; chosen for high accuracy with low compute cost.

Pandas – Efficient data loading, filtering, and CSV output.

Streamlit – Live recruiter-facing sandbox for interactive ranking demo; chosen for zero-config deployment.

GitHub – Version control and public submission hosting.

openpyxl – XLSX export for the ranked output file.

No cloud APIs or LLMs required at runtime – fully reproducible, offline, and deterministic.

Submission Assets

GitHub Repository: https://github.com/datlavenkatarreddy/REDROB_RANKER

→ Full source code, README, requirements.txt, and submission metadata

Live Sandbox (Streamlit): <https://redrobranker-dc9de7oiwr8vyxpjhssxnz.streamlit.app>

→ Interactive recruiter demo – upload candidates and view ranked results instantly

Ranked Output: submission.csv / submission.xlsx

→ Top 100 candidates with rank, score, and plain-English reasoning

Presentation: This deck (PDF submitted via portal)

Validation: Passed official hackathon validator – 100 rows, correct format, sorted by score



INDIA.RUNS

Build what next India runs on

THANK YOU